

Proposal Tugas Besar Pemrograman Berorientasi Objek:

"Monster Inn: Sistem Manajemen Reservasi dan Perawatan Entitas Fantasi Berbasis Pendekatan Pemrograman Berorientasi Objek"



Mata Kuliah:

Pemrograman Berorientasi Objek

Dosen Pengampu:

Lisda S.Kom., M.Kom

Oleh :

Naufal Labib Asydiq	(103112400108)
M.Hanif Al Faiz	(103112400042)
Rafi Maheswara	(103112400135)
Fauza Kaizaku Setiawan	(103112400134)
Gending Lokananta Dwi S.	(103112400250)
Lahra Budi Saputra	(103112430054)
Dzaki Khothir	(103112400113)

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

Daftar Isi

Daftar Isi.....	2
BAB 1.....	4
Pendahuluan.....	4
1.1. Latar Belakang.....	4
1.2. Rumusan Masalah.....	5
1.3. Batasan Masalah.....	5
1.4. Tujuan.....	6
1.5. Manfaat.....	6
1.6. Penjelasan Singkat Aplikasi.....	7
BAB 2.....	8
Pembahasan.....	8
2.1. Analisis Arsitektur Sistem.....	8
2.2. Pemodelan Entitas Sistem.....	9
2.2.1. Identifikasi Entitas.....	9
2.2.2. Analisis Relasi Antar Objek.....	10
2.3. Implementasi Pilar OOP Pada Program.....	11
A. Enkapsulasi (Encapsulation).....	11
B. Abstraksi (Abstraction).....	11
C. Polimorfisme (Polimorfism).....	11
D. Turunan (Inheritance).....	12
2.4. Konstruksi UI dan Fungsionalitas.....	12
2.5. Rincian Distribusi Tugas.....	13
BAB 3.....	15
Penutup.....	15
Daftar Pustaka.....	16

BAB 1

Pendahuluan

1.1. Latar Belakang

Meja resepsionis (*Front Desk*) merupakan pusat saraf operasional dari sebuah hotel sekaligus titik kontak pertama yang menentukan kualitas pengalaman menginap tamu. Dalam operasional sehari-hari, seorang resepsionis dihadapkan pada alur kerja yang serba cepat dan menuntut tingkat akurasi tinggi; mulai dari menangani registrasi tamu (*check-in/check-out*), memantau ketersediaan dan kebersihan kamar (*room status*), melayani permintaan tambahan tamu (*room service*), hingga melakukan rekapitulasi data keuangan saat pergantian *shift* (*night audit*).

Pengembangan sistem *Smart Cashier Desk* pada Monster Inn ini mengadopsi konsep semesta yang terinspirasi dari mekanisme elemen pada game “*Dragon Village*”, di mana setiap entitas memiliki karakteristik unik berdasarkan afiliasi elemennya. Selain itu, logika operasional mengenai kompatibilitas antara tamu dan ruangan mengacu pada sistem tatanan sosial dalam film “*Disney’s Elemental*”, yang menekankan bahwa setiap elemen memerlukan habitat khusus untuk menjaga stabilitas energinya. Secara teknis, kedua referensi ini digunakan sebagai *business rules* (aturan bisnis) utama yang diimplementasikan melalui struktur *Object-Oriented Programming* (OOP) dalam bahasa JavaScript.

Banyak penginapan berskala menengah ke bawah masih mengandalkan sistem manual atau perangkat lunak yang terfragmentasi dalam mengelola tugas-tugas tersebut. Fragmentasi ini sering kali menimbulkan berbagai masalah operasional, seperti antrian panjang saat *check-in* karena proses pendataan yang lambat, miskomunikasi status kamar (misalnya memberikan kunci kamar yang belum dibersihkan kepada tamu), hingga kebocoran pendapatan akibat pesanan layanan kamar yang lupa dicatat ke dalam tagihan akhir tamu. Selain itu, pembuatan laporan harian secara manual memakan waktu lama dan sangat rentan terhadap *human error*.

Untuk mengatasi inefisiensi tersebut, diperlukan sebuah sistem manajemen hotel terpadu yang dirancang khusus dari sudut pandang (POV) resepsionis. Dengan mengimplementasikan arsitektur *Object-Oriented Programming* (OOP), sistem ini dapat memodelkan entitas operasional hotel seperti Tamu, Kamar, Transaksi, dan Layanan ke dalam objek-objek digital yang saling terintegrasi. Pendekatan ini memungkinkan terciptanya sebuah *dashboard* sentral yang mampu mengotomatisasi registrasi, memvisualisasikan status kamar secara *real-time*, mengakumulasi tagihan secara dinamis, dan menghasilkan laporan harian secara presisi dalam satu antarmuka yang cepat dan ergonomis.

1.2. Rumusan Masalah

Berdasarkan latar belakang mengenai kompleksitas penanganan entitas fiktif dan urgensi penerapan arsitektur *Object-Oriented Programming* (OOP) di atas, maka rumusan masalah dalam pengembangan sistem perangkat lunak ini adalah:

1. Bagaimana merancang modul registrasi dan *check-in* berbasis OOP yang tidak hanya mendata informasi entitas tamu (monster), tetapi juga menggunakan algoritma validasi untuk menetapkan kamar (habitat) yang secara presisi kompatibel dengan klasifikasi elemen bawaan tamu tersebut?
2. Bagaimana membangun antarmuka visual yang dapat memetakan dan menampilkan status spesifik dari berbagai tipe kamar ber-elemen (seperti Kamar Lava atau Kamar Akuarium) secara *real-time* guna mencegah terjadinya *problem* alokasi ruang?
3. Bagaimana mengintegrasikan modul layanan tambahan (Room Service) dengan memanfaatkan kelas Monster sebagai antarmuka (*interface*) dan menerapkan konsep Polimorfisme (*Polymorphism*), sehingga tindakan perlakuan spesifik (seperti injeksi magma untuk elemen Api atau nutrisi akuatik untuk elemen Air) dapat dieksekusi secara dinamis dan biayanya terakumulasi secara otomatis ke dalam tagihan akhir tamu?"
4. Bagaimana merancang algoritma rekapitulasi transaksi berbasis objek yang mampu mengekstrak perhitungan dinamis (tarif dasar kamar ditambah biaya perawatan elemen spesifik) menjadi Laporan Pendapatan Harian (*Daily Report*) secara akurat untuk memfasilitasi audit manajerial?

1.3. Batasan Masalah

Agar pengembangan sistem perangkat lunak ini tetap fokus, terarah, dan sesuai dengan alokasi waktu proyek, maka ditetapkan batasan masalah sebagai berikut:

1. **Sudut Pandang Pengguna (POV):** Sistem dirancang secara eksklusif untuk kebutuhan operasional Resepsionis (*Front Desk*). Fitur untuk manajemen tingkat lanjut (*Back-office*, *Human Resource*, atau Inventaris Gudang) tidak termasuk dalam cakupan proyek ini.
2. **Platform & Integrasi:** Purwarupa ini berjalan sebagai aplikasi berbasis web. Untuk fase ini, sistem dikelola secara mandiri (*stand-alone*) dan belum diintegrasikan dengan *Online Travel Agent* (OTA) eksternal seperti Agoda atau Traveloka.
3. **Manajemen Transaksi:** Pembayaran diasumsikan menggunakan pencatatan nominal tunai/transfer secara langsung oleh kasir. Integrasi *Payment Gateway* otomatis pihak ketiga belum diimplementasikan.
4. **Fokus Fitur:** Modul difokuskan pada siklus hidup tamu di hotel: *Check-in*, Pemantauan Kamar, Pemesanan Layanan Tambahan, *Check-out*, dan Pembuatan Laporan Harian.

1.4. Tujuan

Berdasarkan rancangan operasional manajemen hotel dari sudut pandang (POV) resepsionis, proyek pengembangan sistem perangkat lunak ini memiliki tujuan sebagai berikut:

1. **Merancang Modul Registrasi Terpusat:** Membangun sistem pendataan tamu dan pemesanan kamar (*check-in*) yang cepat dan terintegrasi, guna meminimalisir waktu tunggu tamu di meja resepsionis.
2. **Membangun Visualisasi Status Kamar:** Menciptakan antarmuka pemantauan (*Room Management*) yang memungkinkan resepsionis untuk mengecek ketersediaan dan status terkini setiap kamar (Kosong, Terisi, Kotor, atau Perbaikan) secara visual dan *real-time*.
3. **Mengotomatisasi Tagihan Layanan Pelanggan:** Mengimplementasikan fitur *Customer Service* untuk mencatat pesanan *Room Service* atau permintaan tambahan lainnya, di mana sistem secara otomatis mengakumulasi biaya layanan tersebut ke dalam tagihan akhir tamu.
4. **Mengembangkan Fitur Laporan Harian:** Membuat modul rekapitulasi data otomatis (*Daily Report*) yang merangkum metrik operasional harian seperti jumlah tamu masuk/keluar dan total transaksi untuk memudahkan proses tutup buku atau serah terima *shift* resepsionis.
5. **Mengoptimalkan UI/UX untuk Resepsionis:** Mengaplikasikan desain antarmuka pengguna yang ergonomis, intuitif, dan efisien, yang secara khusus disesuaikan dengan alur kerja (POV) staf *Front Desk*.

1.5. Manfaat

1.5.1. Bagi Pihak Manajemen Hotel

- a. **Akurasi Finansial:** Meminimalisir kebocoran pendapatan (*revenue leakage*) yang sering terjadi akibat kelalaian pencatatan layanan tambahan (seperti *Room Service* atau *Laundry*) secara manual.
- b. **Transparansi Operasional:** Memudahkan manajer untuk memantau performa harian hotel melalui laporan pengunjung harian (*Daily Report*) yang dihasilkan secara otomatis dan akurat dari sistem.
- c. **Standarisasi Pelayanan:** Memastikan proses operasional di meja depan berjalan sesuai standar, terstruktur, dan dapat dipertanggungjawabkan (meminimalisir *human error*).

1.5.2. Bagi Staff Resepsionis

- a. **Efisiensi Beban Kerja:** Mengurangi pekerjaan repetitif dan manual, memungkinkan resepsionis untuk bekerja lebih cepat ("sat set") saat melayani antrean tamu yang panjang.
- b. **Pencegahan Miskomunikasi:** Melalui *dashboard* visualisasi status kamar yang *real-time*, resepsionis tidak akan lagi tidak sengaja memberikan kunci kamar yang belum dibersihkan kepada tamu.
- c. **Serah Terima Shift yang Lancar:** Mempermudah proses *Night Audit* atau pergantian jadwal kerja (*shift*), karena data transaksi dan tagihan yang belum lunas sudah terekapitulasi dengan jelas di dalam sistem.

1.5.3. Bagi Tamu Hotel

- a. **Peningkatan Kualitas Layanan:** Tamu merasakan pengalaman *Check-in* dan *Check-out* yang jauh lebih singkat dan tanpa hambatan.
- b. **Tagihan yang Transparan:** Tamu mendapatkan rincian tagihan akhir (*Room Folio*) yang sangat akurat, mencakup seluruh biaya kamar dan tambahan pesanan selama mereka menginap, tanpa ada biaya yang terlewat atau salah hitung.

1.5.4. Bagi Pengembang Akademi

- a. **Penerapan Studi Kasus Nyata:** Menjadi sarana pembuktian yang valid bahwa pilar-pilar *Object-Oriented Programming* (seperti Enkapsulasi dan Polimorfisme) sangat efektif

1.6. Penjelasan Singkat Aplikasi

Sistem Manajemen Hotel ini dirancang sebagai aplikasi terintegrasi berbasis web yang berfungsi sebagai pusat kendali operasional (*Property Management System / PMS*) sekaligus *Point of Sales* (POS). Alur kerja aplikasi secara khusus dirancang untuk mengoptimalkan tugas harian seorang staf resepsionis di meja depan (*Front Desk*).

Operasional dimulai ketika staf resepsionis melakukan autentikasi *login* ke dalam sistem. Setelah kredensial divalidasi, pengguna akan diarahkan ke *dashboard* utama. Dari titik ini, resepsionis dapat melakukan proses *check-in* melalui modul Registrasi Tamu. Dalam proses ini, resepsionis memasukkan data identitas tamu dan mengalokasikan kamar berdasarkan tipe yang diinginkan (misalnya: Air, Api, atau Tanah). Sistem logika di latar belakang secara otomatis akan memvalidasi ketersediaan kamar, lalu memblokir atau menolak pembuatan reservasi jika kamar yang dipilih sedang berstatus terisi (*Occupied*).

Selama tamu menginap, operasional berlanjut pada pemantauan dan pelayanan melalui modul *Customer Service*. Antarmuka ini memungkinkan resepsionis untuk mencatat berbagai permintaan layanan tambahan dari tamu, seperti pesanan makanan (*Room Service*) atau fasilitas pendukung lainnya. Sistem akan memproses instruksi ini dan secara otomatis membebankan biaya layanan tambahan tersebut ke dalam akun tagihan kamar tamu (*Room*

Folio). Pendekatan ini memastikan bahwa setiap pesanan tercatat dengan akurat tanpa memerlukan pencatatan buku besar manual.

Tahap akhir dari siklus operasional adalah proses *check-out*. Melalui modul transaksi, sistem akan mengakumulasi seluruh elemen tagihan secara dinamis. Perhitungan ini mencakup tarif dasar kamar yang dikalikan dengan durasi menginap, ditambah dengan akumulasi biaya layanan tambahan, serta dikurangi deposit uang muka yang telah dibayarkan saat registrasi. Sistem secara cerdas menjalankan kalkulasi akhir ini (menerapkan prinsip Polimorfisme pada objek perhitungan) sehingga mengeliminasi kebutuhan hitung manual oleh kasir. Hasil kalkulasi tersebut kemudian disajikan dalam bentuk nota tagihan elektronik (*Invoice*) yang terperinci, dan data transaksinya akan disimpan secara permanen ke dalam modul Laporan Harian (*Daily Report*) untuk keperluan audit dan serah terima *shift*.

BAB 2

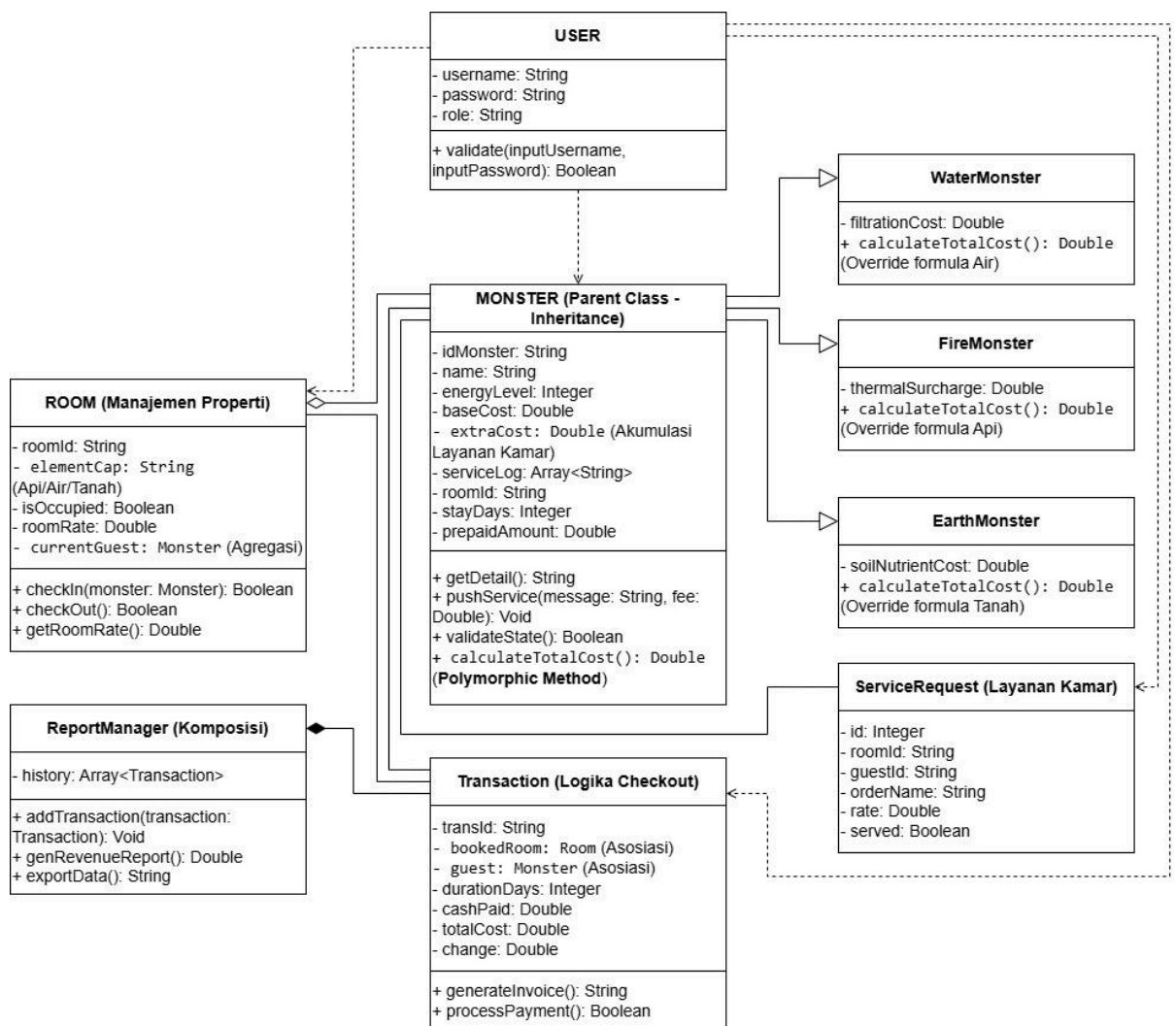
Pembahasan

2.1. Analisis Arsitektur Sistem

Sistem Monster Inn dirancang sebagai purwarupa aplikasi kasir (*Smart Cashier Desk*) yang mengintegrasikan logika operasional hotel dengan aturan semesta naga berelemen. Proyek ini menggunakan arsitektur **Single Page Application (SPA)** berbasis JavaScript, di mana semua interaksi dikelola dalam satu halaman untuk efisiensi kerja resepsionis.

2.2. Pemodelan Entitas Sistem

Perancangan sistem didasarkan pada kebutuhan pengelolaan tamu yang memiliki karakteristik habitat yang spesifik, terinspirasi dari mekanisme elemen pada *Dragon Village* dan *Disney's Elemental*.



2.2.1. Diagram UML

Berikut adalah tabel identifikasi kelas yang digunakan untuk membangun sistem ini setelah penyederhanaan fitur:

Nama Class	Peran / Deskripsi	Atribut Kunci (Data)	Method Utama (Perilaku)
User	Mengelola data autentikasi dan peran staf resepsionis.	username, password, role	validate()
Monster	Superclass (Induk) yang menyimpan data dasar tamu hotel.	idMonster, name, baseCost, extraCost, serviceLog	getDetail(), pushService(), calculateTotalCost()
FireMonster	Turunan Monster khusus	thermalSurcharge	calculateTotalCost()

	elemen Api dengan biaya termal tambahan.	element: "Api"	(Override), feedElement()
WaterMonster	Turunan Monster khusus elemen Air dengan biaya filtrasi tambahan.	filtrationCost, element: "Air"	calculateTotalCost() (Override), feedElement()
EarthMonster	Turunan Monster khusus elemen Tanah dengan biaya nutrisi tambahan.	soilNutrientCost, element: "Tanah"	calculateTotalCost() (Override), feedElement()
Room	Mengelola status ketersediaan kamar dan validasi check-in.	roomId, elementCap, isOccupied, currentGuest	checkIn(), checkOut(), getRoomRate()
Transaction	Mengkalkulasi tagihan akhir dan memproses pembayaran saat checkout.	transId, bookedRoom, guest, durationDays, totalCost	generateInvoice(), processPayment()
ReportManager	Mengumpulkan riwayat transaksi untuk laporan pendapatan harian.	history (Array of Transaction)	addTransaction(), genRevenueReport()

2.2.2. Identifikasi Entitas

Sistem ini menerapkan berbagai jenis relasi untuk memastikan integritas data:

1. **Inheritance:** Sub-kelas elemen mewarisi seluruh atribut dan metode dari kelas Monster, memungkinkan penggunaan kembali kode (*code reuse*) yang efisien.
2. **Aggregation:** Kelas Room memiliki hubungan agregasi dengan Monster melalui atribut currentGuest, di mana objek tamu dapat dilepaskan dari objek kamar tanpa menghancurkan data tamu tersebut.
3. **Composition:** ReportManager memiliki hubungan komposisi dengan Transaction, di mana setiap riwayat transaksi merupakan bagian mutlak dari laporan harian yang dikelola.

2.3. Implementasi OOP Pada Program

Implementasi kode pada script.js menerapkan pilar-pilar OOP untuk menangani kompleksitas layanan elemen secara otomatis.

A. Enkapsulasi (Encapsulation)

Sistem melindungi data krusial melalui metode akses terbatas.

1. **Keamanan Login:** Atribut password pada kelas User bersifat internal dan hanya bisa diverifikasi melalui metode `validate()`.
2. **Integritas Tagihan:** Biaya tambahan tamu tidak diubah secara manual, melainkan melalui metode `pushService()` yang sekaligus mencatat histori ke dalam `serviceLog`.

B. Abstraksi (Abstraction)

Penerapan prinsip abstraksi pada sistem Monster Inn bertujuan untuk menyembunyikan kompleksitas logika internal dari pengguna (resepsionis) dan hanya menampilkan fungsi yang esensial untuk operasional.

1. **Kelas Monster sebagai Kontrak:** kelas Monster berperan sebagai *Abstract Base Class* yang mendefinisikan "kontrak" perilaku bagi seluruh tamu naga. Metode seperti `calculateTotalCost()` didefinisikan secara abstrak di kelas induk untuk memastikan setiap elemen memiliki cara perhitungan biayanya sendiri.
2. **Penyembunyian Kerumitan Rumus:** Staf resepsionis tidak perlu memahami variabel teknis yang berbeda di setiap elemen, seperti `thermalSurcharge` (Api) atau `filtrationCost` (Air). Staf cukup memanggil satu metode tunggal, yaitu `calculateTotalCost()`, dan sistem secara otomatis mengolah variabel-variabel tersebut di latar belakang untuk menghasilkan nilai tagihan akhir yang akurat.
3. **Abstraksi Layanan melalui `pushService()`:** Proses pembaruan saldo tagihan dan pencatatan riwayat layanan disederhanakan melalui metode `pushService()`. Resepsionis cukup memasukkan nama layanan dan tarifnya, sementara detail mengenai bagaimana biaya tersebut diakumulasikan ke dalam atribut `extraCost` dan bagaimana pesan disusun ke dalam *array* `serviceLog` ditangani sepenuhnya oleh logika internal objek.

C. Polimorfisme (Polimorphism)

Pilar Polimorfisme pada sistem Monster Inn diimplementasikan melalui mekanisme *Method Overriding*, di mana sub-kelas (elemen) memberikan implementasi spesifik untuk metode yang telah didefinisikan pada kelas induk (*Monster*).

1. Polimorfisme pada Kalkulasi Biaya

Metode `calculateTotalCost()` adalah contoh utama penerapan polimorfisme dinamis dalam sistem ini. Staf resepsionis cukup memanggil satu metode yang sama saat *checkout*, namun sistem akan menjalankan rumus yang berbeda di latar belakang sesuai dengan objek elemen yang aktif:

1. **FireMonster:** Menggunakan rumus biaya dasar ditambah beban termal (`thermalSurcharge`).
2. **WaterMonster:** Menggunakan rumus biaya dasar ditambah biaya filtrasi (`filtrationCost`).
3. **EarthMonster:** Menggunakan rumus biaya dasar ditambah nutrisi tanah (`soilNutrientCost`).

2. Polimorfisme pada Layanan Perawatan Habitat

Sistem mengintegrasikan layanan tambahan melalui metode yang didefinisikan sebagai "kontrak" pada kelas Monster. Polimorfisme memungkinkan eksekusi tindakan yang berbeda secara otomatis:

1. **Eksekusi Dinamis:** Saat resepsionis menjalankan perintah `feedElement()`, sistem secara cerdas memberikan **Injeksi Magma** (Rp 60.000) untuk elemen Api, namun memberikan **Nutrisi Akuatik** (Rp 70.000) jika tamu tersebut adalah elemen Air.
2. **Akumulasi Biaya Otomatis:** Setiap tindakan polimorfik ini memanggil metode `pushService()`, yang secara otomatis mengakumulasikan biaya layanan ke dalam atribut `extraCost` tanpa perlu campur tangan manual dari staf kasir.

D. Turunan (Inheritance)

Pilar *Inheritance* menciptakan hierarki yang memungkinkan kelas elemen (FireMonster, WaterMonster, EarthMonster) mewarisi atribut dasar dari kelas induk Monster menggunakan kata kunci `extends`. Penggunaan fungsi `super()` pada konstruktor kelas anak memastikan properti dasar tamu terinisialisasi dengan benar sebelum atribut spesifik elemen ditambahkan.

2.4. Konstruksi UI dan Fungsionalitas

Antarmuka sistem dirancang berbasis *dashboard* untuk memfasilitasi kerja resepsionis agar lebih cepat dan terpusat. Berikut adalah pemetaan layar antarmuka beserta fungsinya:

Nama Antarmuka	Elemen Visual Utama	Fungsi Utama	Class Terkait
Login Screen	Form kredensial, Tombol Login	Mengautentikasi staf resepsionis untuk mengakses dashboard operasional.	Receptionist
Dashboard Registrasi	Form biodata tamu, Dropdown tipe kamar, Input deposit uang	Memproses Check-in, mendaftarkan tamu baru, memilih kamar kosong, dan mencatat deposit.	Guest, Room, Reservation
Dashboard Kamar	Grid/Card visual dengan kode warna (Hijau/Merah/Kuning)	Memantau ketersediaan dan merubah status kamar (Available, Occupied, Dirty, Maintenance) secara visual.	Room
Dashboard Layanan	Tabel antrean layanan (Pending/Completed), Tombol aksi	Mencatat pesanan tamu (Room Service) dan otomatis	ServiceRequest, Reservation

		membebankan biaya tersebut ke tagihan kamar.	
Dashboard Checkout	Dropdown tamu aktif, Rincian nota (Invoice Preview)	Memproses Check-out, mengkalkulasi total tagihan dikurangi deposit, dan mencetak nota akhir.	Reservation, Room
Dashboard Laporan	Kartu metrik (KPI), Tabel riwayat transaksi harian	Menampilkan rekapitulasi data operasional (total omset, jumlah tamu) untuk proses serah terima shift.	DailyReport, Reservation

2.5. Rincian Distribusi Tugas

Nama	Front-End (UI/UX)	Back-End (OOP)
Naufal Labib	Login Screen & validasi input.	Class User: Enkapsulasi, kredensial login, dan validasi akses.
M. Hanif	Dashboard Kamar (Grid/Table real-time).	Class Room: Agregasi tamu, metode checkIn() & checkOut().
Fauza Kaizaku	Dashboard Registrasi & input biodata.	Class Monster (Parent): Inheritance & metode <i>abstract</i> .
Rafi Maheswara	Komponen Dinamis (Deteksi elemen monster).	Sub-class Elemen: Polimorfisme & <i>Overriding</i> calculateTotalCost().
Dzaki Khothir	Dropdown tamu aktif, Rincian nota (Invoice Preview)	Class ServiceRequest: Relasi objek untuk akumulasi biaya tambahan.
Gending Lokananta	Dashboard Layanan (Room Service & Special Care).	Class Transaction: Logika finansial, asosiasi, & <i>Exception Handling</i> .
Lahra Budi S	Dashboard Checkout & Invoice Preview.	Class ReportManager: Komposisi data & rekapitulasi omset.

BAB 3

Penutup

3.1. Kesimpulan

Berdasarkan pemaparan di atas, dapat disimpulkan bahwa pengembangan sistem Monster Inn merupakan solusi inovatif untuk menjawab tantangan operasional pada meja resepsionis melalui pendekatan Object-Oriented Programming (OOP). Dengan mengintegrasikan pilar-pilar utama seperti enkapsulasi untuk keamanan data, serta polimorfisme untuk otomatisasi kalkulasi biaya layanan berbasis elemen, sistem ini mampu menyederhanakan kompleksitas manajemen entitas fantasi ke dalam satu antarmuka yang efisien.

Implementasi sistem ini diharapkan dapat memberikan dampak signifikan, baik dalam mengurangi beban kerja manual staf resepsionis maupun dalam menjamin akurasi finansial bagi pihak manajemen hotel. Dengan visualisasi status kamar secara real-time dan sistem penagihan yang transparan, Monster Inn tidak hanya berfungsi sebagai alat bantu teknis, tetapi juga sebagai standar baru dalam meningkatkan kualitas pengalaman menginap bagi para tamu. Akhir kata, proyek ini menjadi sarana pembuktian akademis bahwa struktur pemrograman yang solid dapat menghasilkan sistem manajemen properti yang cerdas, adaptif, dan siap menghadapi kebutuhan operasional yang kompleks.

LAMPIRAN

Link Google Drive :

https://drive.google.com/drive/folders/1z1Vzh-6lWQJoEDqTWK_0k6tNG8ko9XkJ?usp=sharing

Daftar Pustaka

- Bristy, I. J. (2025). Smart Reservation and Service Management Systems: Leveraging MIS for Hotel Efficiency. Lamar University - Management Information Systems. 1
- Efan, Krismadinata, Jama, J., & Mulya, R. (2023). Object-Oriented Programming Course: A Systematic Literature Review on Problems and Solutions. International Journal of Information and Education Technology, 13(2), 1808-1815. 2
- Hemsley, C. (2025). The Case for a Polymorphic Database: Bringing the Flexibility of Code to Your Data Architecture. TypeDB Research / Software Engineering Data Layer Analysis. 6
- Sihotang, W. (2024). innovative Strategy for Digital Business Development Based on Object-Oriented Programming to Improve System Efficiency and Scalability. Business and Information Management (BIM) Journal. 5
- Wyrich, M., Hofmeister, J. C., Apel, S., & Siegmund, J. (2024). Object Disorientation: Uncovering the Essence of Object-Oriented Programming. IEEE Software / IEEE Xplore. 3
- (2024/2025). Exploring Polymorphism Flexibility and Code Reusability in Object-Oriented Programming. ResearchGate Publication / Software Engineering Analysis. 4